CSCI212: Lecture #17

Kevin Bedoya

Wednesday **4/13/26**

Objective of today's lecture

- How do we draw pictures in **swing**
- We override (happens when a subclass redefines a superclass method) the **paintComponent** method. Swing components come with generic paintComponent methods. Our plan is to use subclasses that override this method.

When swing runs it decides at certain times that the display should update. When this happens, it runs the paintComponent method on swing objects in the display. If our components are standard swing classes they are drawn in the standard way. If we make subclasses t0 override paintComponent they are shown our way.

What swing component can we override?

- Any with a paint component method.
- Some like **JFrame**, **JButton** are a bad idea to use, but **JPanel** and **JComponent** are good choices.

What do we need to draw an image?

- We'll need a **JFrame** that serves as a canvas for our image
- We'll need a **JPanel** to paint a shape

```
1  public class SimpleDrawing extends JPanel {
2      //override paintComponent
3      void printComponent(Graphics g){
4          g.setColor(Color.BLUE);
5          // the coordinates are relative to the top left of the rectangle
6          // width by height pixel dimensions
7          g.drawRect(x,y, width, height);
8      }
9  }
```

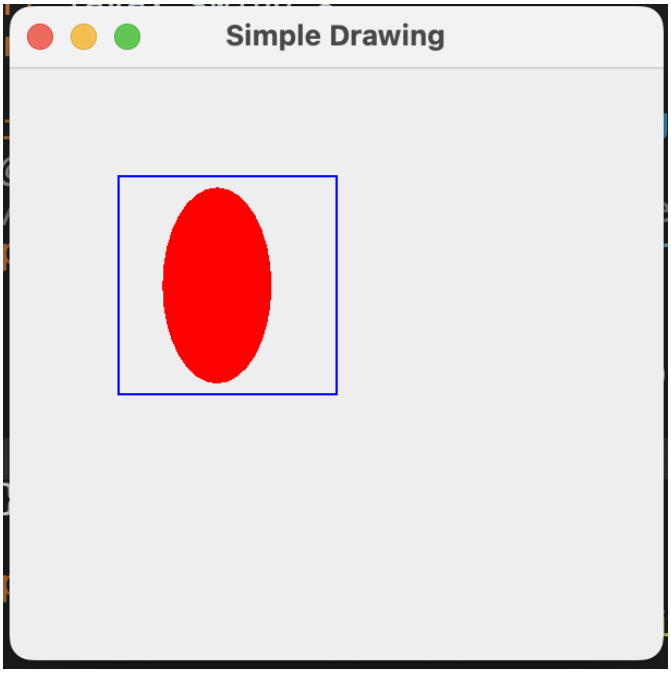
Shape drawing demo

```java
import javax.swing.*;
import java.awt.*;

public class SimpleDrawing extends JPanel{
    @Override
    // You place an annotation to let Java know what you're doing (it's optional)
    protected void paintComponent(Graphics g){
        super.paintComponent(g);
        g.setColor(Color.BLUE);
        g.drawRect(50, 50, 100, 100);
        g.setColor(Color.RED);
        g.fillOval(70, 55, 50, 90);
    }

    public static void main(String[] args){
        SimpleDrawing panel = new SimpleDrawing();

        JFrame frame = new JFrame("Simple Drawing");
        frame.add(panel);
        frame.setSize(300, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true); //Swing will run paintComponent, and draw everything onto the screen.
    }
}
```



Review on significant Java keywords

- **Super** - refers to the superclass of a class; gives us access to the actual methods of the super class.

```
    // e.g.
    super.paintComponent(g); // cleans the screen and initializes new drawings.

```

- **Protected** - protected attributes of a class are only accessible within the class. Subclasses can inherent protected attributes of superclasses; protected attributes are only accessible within subclasses.

The drawing app

```
import javax.swing.*;
import java.awt.*;

public class DrawingApp {
    public static void main(String[] args){

        // Model: holds shape state
        ShapeModel model = new ShapeModel();

        // View: renders based on model
        DrawingPanel panel = new DrawingPanel(model);

        // Control: user interaction
        JButton button = new JButton("Increase Width");

        button.addActionListener(e -> {
            model.increaseWidth(10);  // update model
            panel.repaint();          // refresh view
        });

        // Frame setup
        JFrame frame = new JFrame("Drawing with Model");
        frame.setLayout(new BorderLayout());

        frame.add(panel, BorderLayout.CENTER);
        frame.add(button, BorderLayout.SOUTH);

        frame.setSize(400, 400);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Shape Model

```java
public class ShapeModel {
    private int width = 50;
    private int height = 100;

    // Accessors
    public int getWidth(){
        return width;
    }

    public int getHeight(){
        return height;
    }

    // Mutator (missing in original)
    public void increaseWidth(int delta){
        width += delta;
    }
}
```

Drawing Panel

```java
import javax.swing.*;
import java.awt.*;

public class DrawingPanel extends JPanel {
    private ShapeModel model;

    public DrawingPanel(ShapeModel model){
        this.model = model;
    }

    @Override
    protected void paintComponent(Graphics g){
        super.paintComponent(g); // important: clears previous drawings

        // Draw using current model state
        g.setColor(Color.RED);
        g.fillOval(50, 50, model.getWidth(), model.getHeight());
    }
}
```